Validation status

This delivery is source, not a verified Windows binary. The Unity Editor and Windows runtime were not installed or run in the Linux delivery environment. No Unity gameplay screenshot, Windows performance figure, or engine compilation success is claimed.

Checks actually executed

- **62 static checks passed**, covering syntax parsing of all seven Unity C# files, unfinished-code marker checks, ten readable transparent sprites, ten valid PCM WAV files, asset-reference names, package/editor pins, legacy input selection, both gamepad axes, generation hooks, prefab wiring, and Windows x64 build target.
- **21 engine-independent C# tests passed**, compiled with Mono `mcs` and executed with `mono`: campaign counts, health/score rules, capped repairs, clamping, swept projectile hit/miss, stationary segments, boundary contacts, out-of-segment misses, and relative-motion collision geometry.
- Both Python tools passed Python bytecode compilation.
- The bundled sprite contact sheet was visually inspected. It contains eight illustrated game assets plus neutral projectile/particle primitives with transparent backgrounds. This is **asset inspection, not a render of the running game**.
- Unity 6000.0.62f1 release/revision was checked against its official release page. The build-target support and NamedBuildTarget/Mono2x API signatures were checked against Unity 6 documentation.

Evidence: `StaticChecks.json`, `RulesTestResults.txt`, `AssetPreview.png`.

Not verified here

- Full Unity C# compilation and module resolution.
- Initial asset import, SpriteRenderer shader/material defaults, generated scene/prefab serialization, Editor startup callbacks, and automatic input-axis configuration.
- Actual gameplay balance, campaign completion, hit feel, audio mix/playback, display resizing, pause/focus behavior, or gamepad mappings.
- Windows player build, player launch, and persistent PlayerPrefs on Windows.

The largest integration risks are Unity’s first-import defaults/schema normalization and device-specific legacy gamepad indices. The project pins its Editor, uses documented Editor serialization APIs, avoids hand-authored scene/prefab YAML, and provides a rebuild menu to make these steps inspectable. Static parsing cannot prove that Unity resolves every API or serialized property; any Console error needs its exact message checked in the pinned Windows Editor.

Windows acceptance checklist — not yet executed

1. Import in 6000.0.62f1. Confirm no red Console errors, ten PNG sprite assets, ten WAV clips, nine generated prefabs, GameAssets, and Starfall.unity in Scene List.
2. Enter Play mode. Launch by clicking and by Enter. Move with WASD and arrows; diagonal speed should match cardinal speed. Hold Space and mouse fire independently.
3. Confirm friendly shots only damage enemies, enemy shots only damage the player, hull decreases on contact, invulnerability flashes briefly, and an escaped enemy costs one hull even with a shield.
4. Observe straight, aimed, fan, and boss radial fire. Each wave advances only when its remaining ships are gone; enemy bullets clear at intermission.
5. Collect each capsule: repair adds two without exceeding five; triple shot lasts twelve seconds; shield lasts eight. Buff timers freeze while paused.

6. Escape pauses/resumes. Alt-tab pauses. Restart clears the field, score, buffs and campaign state. Repeat restart several times; no extra starfield/audio instances or duplicate controllers should accumulate.
7. Lose all hull and test Replay and Main Menu. Complete the boss and test the sector-clear screen. Exit and relaunch to confirm best score/audio preference persistence.
8. Resize the game window, check all HUD/menu text fits, verify three star layers move at different speeds and audio mute works. Listen for each action sound and a smooth drone loop.
9. With an Xbox-compatible gamepad, test stick, A/B/X/Y, and Start. Remap PilotInput if this device reports a different legacy button order.
10. Build Windows x64 from the Starfall menu. Run the executable outside Unity and repeat launch, movement, firing, pause, game-over/restart, mute, resize, quit, and persistence checks. Ship the entire build folder only after this succeeds.